Beyond Algorithms:

Hardware-Constrained Data Systems from Storage to Search

A Comparative Review of Three Systems Papers

by

Xixian QI

120090691

Final Report
Prepared for the Operating Systems course
in partial fulfillment of the course requirements

School of Data Science
The Chinese University of Hong Kong, Shenzhen

May 2026

Abstract

Modern data-intensive applications must store, process, and search massive datasets under tight latency and cost constraints. This report studies three systems papers through one organizing argument: **data movement, locality, and scheduling granularity form a common optimization substrate across storage, relational processing, and vector search systems**. Algorithms designed only for an abstract RAM model often fail to exploit heterogeneous storage tiers, GPU memory hierarchy, and GPU scheduling behavior; hardware-conscious co-design makes these constraints explicit in the algorithm and system architecture.

The report compares three focal papers. HiStore uses cost-model-driven SSTable placement across NVMe SSD, SATA SSD, and HDD tiers for LSM-tree key–value stores, improving throughput by 1.3×–2.1× and reducing P99 latency by 2.5×–4.1×. A GPU hash join with partition pruning uses lightweight hash histograms to skip empty partition pairs, achieving up to 4.7× speedup on selective joins while improving memory coalescing from 42% to 89%. Hitcher reorganizes GPU vector search around cluster-centric execution and hitch-ride scheduling, reducing P50 latency by 1.9×–2.7× and P99 latency by 2.8×–4.3× compared with state-of-the-art GPU ANN systems.

Across the three papers, the optimization unit shifts from SSTable to partition pair to graph cluster, while the bottleneck shifts from storage tier latency to GPU memory divergence to kernel launch and global-memory overhead. The comparison shows that algorithm–hardware co-design is not a single-system trick, but a reusable design principle for data systems.

Keywords: hardware-conscious data systems, GPU acceleration, key–value stores, hash join, vector search, approximate nearest neighbor, LSM-tree, heterogeneous storage, data movement optimization, co-design

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Modern applications must store, process, and search massive datasets under tight latency and cost constraints. In this setting, the limiting factor is often not algorithmic expressiveness but hardware efficiency: data placement, memory movement, and scheduling overhead determine whether a theoretically efficient algorithm performs well in practice.

This shift is driven by hardware heterogeneity. A single deployment may combine DRAM, NVMe SSDs, SATA SSDs, HDDs, GPUs, and high-speed interconnects, each with different latency, bandwidth, capacity, and cost profiles. Algorithms optimized only for the abstract RAM model can perform poorly on such platforms, while algorithms co-designed with hardware constraints can unlock much larger gains.

This report reviews three systems papers through the lens of **algorithm–hardware co-design**: designing algorithms and system mechanisms together around the hardware bottleneck they must overcome. The three papers cover three layers of the data systems stack:

1. **The storage layer:** How should a key–value store manage data placement across a heterogeneous storage hierarchy (DRAM, NVMe SSD, SATA SSD, HDD) to balance performance and cost?

2. **The processing layer:** How can GPU accelerators be effectively harnessed for relational join processing, given their limited memory capacity and sensitivity to irregular memory access patterns?

3. **The search layer:** How can GPU-based vector search achieve both high throughput and low latency when processing high-dimensional vector data under strict service-level objectives?

Together, these questions test whether hardware-conscious design is a reusable principle across storage, processing, and search.

1.2 Research Questions

Beyond the per-layer challenges above, this report is organized around three overarching research questions that unify the papers and provide analytical axes for cross-paper comparison:

RQ1: *How should data systems expose hardware constraints as first-class optimization variables?* Current systems treat hardware characteristics as opaque—queries are optimized for logical cost models, and placement decisions use static heuristics. This research question asks whether making hardware constraints (latency tiers, memory hierarchy, SIMT semantics, kernel launch overhead) explicit in the system's cost model yields consistent performance improvements.

RQ2: *How does the optimization granularity change from storage placement to GPU execution scheduling?* The three papers operate at different granularities: SSTable-level placement (storage), partition-pair-level pruning (processing), and cluster-level scheduling (search). This research question asks whether there is a coherent framework that explains how the optimization unit should be chosen relative to the hardware constraint being addressed.

RQ3: *When does hardware-conscious co-design outperform algorithm-only or hardware-only approaches?* Algorithmic improvements and faster hardware each help, but they do not remove all hardware-induced waste. This question asks when co-design provides gains that neither approach alone can achieve.

These research questions are revisited in Chapter 6, where we provide cross-paper evidence and a synthesized answer to each.

1.3 Report Scope and Paper Selection

This report focuses on *hardware-conscious data systems*: systems whose core algorithms and data structures are co-designed with, rather than merely deployed on, heterogeneous hardware. The scope spans three layers of the data processing stack—storage, relational processing, and vector search—chosen because they represent the dominant computational patterns in modern data-intensive applications (I/O-bound scan, compute-bound join, and latency-bound search, respectively).

The three papers were selected to form a coherent narrative along two dimensions. *Vertically*, they ascend the systems stack from storage to search, suggesting that the co-design principle is layer-independent. *Horizontally*, each paper addresses a different hardware bottleneck (storage tier latency, GPU memory divergence, kernel launch overhead), suggesting that co-design is bottleneck-independent.

1.4 Challenges

Building hardware-constrained data systems presents a set of interconnected challenges that span storage architecture, parallel computation, and real-time query processing.

1.4.1 Challenge 1: Heterogeneous Storage Management

Storage tiers differ by orders of magnitude in latency, bandwidth, capacity, and price. A cost-effective key–value store must use fast tiers for hot data and cheap tiers for cold data, but common tiering policies are static or cache-like. The challenge is to make placement and migration aware of device characteristics, workload skew, and the LSM-tree structure.

1.4.2 Challenge 2: GPU-Accelerated Relational Processing

GPUs can accelerate hash joins, but only when data movement and memory divergence are controlled. Limited GPU memory, PCIe transfer cost, and SIMT execution all punish naive partitioning. The challenge is to prune useless work and shape active work so GPU threads access memory coherently.

1.4.3 Challenge 3: Latency-Constrained GPU Vector Search

GPU ANN search must balance throughput and latency. Batching improves GPU utilization but increases tail latency; per-query execution reduces waiting but loses data reuse. The challenge is to reuse graph data across queries without forcing queries to wait for large batches.

1.5 Focal Papers

This report studies three focal papers, shown in Figure 1.1.

1.5.1 Paper 1: Intelligent Hybrid Storage for Key–Value Stores (Chapter 3)

The first paper studies **HiStore**, a hybrid storage architecture for LSM-tree key–value stores. HiStore uses a per-SSTable cost–benefit model, integrates placement with compaction, and adapts placement as workloads shift. The result is a better throughput–latency–cost trade-off than static tiering.

This paper illustrates the **storage-layer case**: data placement can be treated as a first-class optimization problem co-designed with the storage engine's internal data structures, not as an orthogonal caching concern.

1.5.2 Paper 2: Partition-Pruning GPU Hash Join (Chapter 4)

The second paper addresses the GPU-accelerated relational processing challenge through a **GPU-based hash join algorithm with partition pruning**. The method eliminates unnecessary data movement and computation by identifying partition pairs that cannot produce join results. A lightweight pre-pass over hash value distributions prunes many partitions, especially for selective joins common in analytical workloads. The pruned partition information is then exploited by a co-designed GPU kernel that coalesces memory accesses to active partitions, reducing global memory transactions and improving SIMT utilization. The technique is extended to multi-GPU configurations via shared pruning metadata. Evaluation shows that partition pruning yields substantial speedups over state-of-the-art GPU hash join implementations, with the greatest benefits on low-selectivity joins where empty-partition overhead dominates.

This paper illustrates the **processing-layer case**: GPU acceleration of relational operators is most effective when complemented by data-reduction techniques that explicitly account for GPU memory constraints and SIMT execution semantics.

1.5.3 Paper 3: Cluster-Centric GPU Vector Search (Chapter 5)

The third paper addresses the latency-constrained GPU vector search challenge through **Hitcher**, a GPU-based vector search system featuring two mechanisms. First, a **Cluster-Centric Kernel** inverts the conventional query-first execution model: instead of assigning one thread block per query, Hitcher groups queries by their target graph cluster and assigns one thread block per cluster, loading cluster vectors into GPU shared memory once and reusing them across all queries targeting that cluster. This reduces redundant global memory reads in query-centric designs. Second, **Hitch-Ride Ordering** replaces batch-based query accumulation with a continuous stream model where incoming queries immediately "hitch a ride" on already-active clusters, eliminating batch accumulation latency while preserving the throughput benefits of shared data access. Hitcher is further optimized with strategic CPU offloading of straggler clusters and asynchronous multi-GPU data transfer. Evaluated on standard ANN benchmarks, Hitcher reduces both median and tail latency by up to several times compared to state-of-the-art GPU ANN systems including Faiss and Rummy, while matching or exceeding their throughput. The paper was accepted at SIGMOD 2026 (to appear).

This paper illustrates the **search-layer case**: applying hardware-conscious co-design principles to vector search can improve latency and throughput beyond what either algorithmic improvements or hardware acceleration alone would suggest.

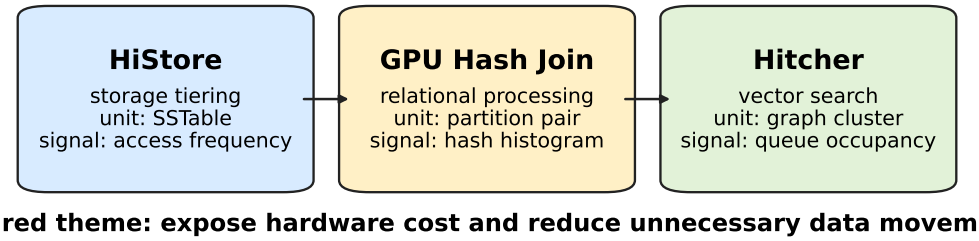

Figure 1.1: Report structure across storage, processing, and search.

1.6 Cross-Paper Interrelations

The three papers share two axes:

- **Data movement reduction:** All three papers target unnecessary data movement, but at different levels of the memory hierarchy. HiStore reduces cross-tier storage I/O by promoting hot data to fast devices. The GPU hash join reduces intra-GPU memory traffic by pruning empty partitions and coalescing access to active ones. Hitcher reduces redundant global memory reads by sharing cluster data across queries in shared memory.
- **Locality-aware scheduling:** Each paper reorganizes computation to exploit hardware locality. HiStore exploits storage locality by co-locating hot data on fast tiers. The GPU hash join exploits memory locality by restructuring thread–partition assignments for coalesced access. Hitcher exploits GPU shared memory locality through cluster-centric scheduling that maximizes data reuse within a thread block.

Cost-performance modeling provides the quantitative backbone: each paper makes the hardware trade-off explicit and tunable.

1.7 Report Organization

The remainder of this report is organized as follows.

Chapter 2 provides the shared background and analytical framework used in the cross-paper comparison.

Chapters 3, 4, and 5 review the three focal papers using a common structure: motivation, design, evaluation, limitations, and summary.

Chapter 6 synthesizes the three papers and answers the research questions.

Chapter 7 concludes with future directions.

Chapter 2

Background and Unified Analytical Framework

This chapter serves two purposes. Sections 2.1–2.4 provide the technical background necessary for the three focal papers. Sections 2.5–2.5.4 establish a unified analytical framework—including hardware cost dimensions, optimization unit taxonomy, evaluation metrics, and threats to cross-paper comparison—that structures the cross-paper synthesis in Chapter 6.

2.1 LSM-Tree Storage Engines

Log-Structured Merge Trees (LSM-trees) [1] are the dominant storage engine architecture for write-intensive key–value stores, used by systems including RocksDB [2], LevelDB [3], and Cassandra [4]. An LSM-tree organizes data into a hierarchy of levels: writes are first buffered in an in-memory *memtable*; when the memtable fills, it is flushed to disk as an immutable *SSTable* (Sorted String Table) at level 0. Background *compaction* merges SSTables from level L into level $L+1$, maintaining sorted order and removing duplicates and tombstones.

The key performance parameters of an LSM-tree are:

- **Write amplification:** the ratio of total bytes written (including compaction) to logical bytes written, typically 10–50× depending on the size ratio and compaction strategy.
- **Read amplification:** the number of SSTables that must be checked per point lookup, bounded by the sum of level sizes divided by the Bloom filter false-positive rate.
- **Space amplification:** the ratio of physical storage used to logical data size, driven by temporary duplication during compaction.

SSTables are the fundamental data placement unit in LSM-trees. Each SSTable is a sorted, immutable file containing key–value pairs, an index block, and a Bloom filter. In

RocksDB, SSTables at different levels may reside on different storage devices, but the default placement policy assigns devices by level number, not by access frequency—the gap that HiStore (Chapter 3) addresses.

2.2 GPU Architecture and Programming Model

Modern GPUs such as the NVIDIA A100 provide massive parallelism through thousands of cores organized into streaming multiprocessors (SMs). The A100 [5] features 108 SMs, each with 64 FP32 CUDA cores and 192 KB of shared memory, delivering up to 19.5 TFLOPS of FP32 compute and 1.5–2.0 TB/s of memory bandwidth from 40–80 GB of HBM2e.

The GPU memory hierarchy is critical for algorithm design:

- **Global memory:** large (40–80 GB) but high latency (~200–800 cycles). Access is most efficient when consecutive threads access consecutive addresses (*memory coalescing*).
- **Shared memory:** small (up to 164 KB/SM on A100) but low latency (~20–30 cycles). Explicitly managed by the programmer, used for data reuse within a thread block.
- **Registers:** fastest but most limited (~255 per thread on A100). Excessive register usage limits occupancy.

GPU threads are organized into *warps* of 32 threads that execute in lockstep (SIMT model). *Warp divergence*—where threads in the same warp take different control-flow paths—serializes execution and reduces utilization. Similarly, uncoalesced memory access patterns serialize global memory transactions, degrading effective bandwidth.

Kernel launch overhead is the fixed cost of dispatching a GPU kernel from the CPU, typically 2–10 µs on modern hardware. For fine-grained operations, this overhead can dominate execution time, motivating techniques that batch or stream kernel launches.

CUDA streams allow concurrent kernel execution and asynchronous memory transfer. Multi-GPU systems connected via NVLink (600 GB/s bidirectional on A100) enable data parallelism but introduce cross-GPU synchronization costs.

2.3 Hash Join Algorithms

Hash join is the dominant algorithm for equi-join in modern query processors. The canonical algorithm consists of two phases:

1. **Build phase:** scan the smaller relation (build relation R) and insert each tuple into a hash table keyed on the join attribute.

2. **Probe phase:** scan the larger relation (probe relation S) and look up each tuple's join key in the hash table; matching pairs form join results.

For GPU acceleration, the hash join is typically *partitioned*: both relations are hash-partitioned on the join key into P partitions, and only partitions R_i and S_i (with the same hash value range) need to be joined. This enables parallel execution across partitions and fits within GPU memory constraints.

A critical inefficiency in partitioned GPU hash joins is *empty partition overhead*: when the join is selective, many partition pairs (R_i, S_i) may produce zero results because R_i or S_i is empty. Yet naive implementations still allocate GPU threads and memory for these empty partitions, wasting both computation and bandwidth—the problem that the GPU hash join in Chapter 4 addresses.

2.4 Graph-Based Approximate Nearest Neighbor Search

Approximate nearest neighbor (ANN) search finds the k closest vectors to a query vector under a distance metric, with a configurable accuracy–speed trade-off. Graph-based ANN indexes construct a *proximity graph* over the vector dataset, where each vertex represents a vector (or cluster of vectors) and edges connect nearby vectors.

At query time, a *greedy traversal* starting from one or more entry points visits vertices in order of proximity to the query, expanding the candidate set until convergence. The most influential graph-based indexes include:

- **HNSW** [6]: Hierarchical Navigable Small World graph with multi-layer navigation, offering state-of-the-art recall–latency trade-offs on CPU.
- **NSG** [7]: Navigating Spreading-out Graph that enforces monotonic search paths, reducing unnecessary distance computations.
- **DiskANN / Vamana** [8]: Disk-based index designed for billion-scale datasets, using a compressed representation for SSD-resident vectors.
- **Faiss-GPU** [9]: IVF-PQ index with GPU acceleration, primarily optimizing for throughput via batching.

GPU deployment of graph-based ANN faces the throughput–latency tension described in Challenge 3 (Section 1.4.3), which Hitcher (Chapter 5) resolves.

2.5 A Unified Model of Hardware-Conscious Optimization

The three papers can be understood as instances of a single optimization pattern:

$$\min_{\text{placement / schedule}} \sum_{\text{data unit}} \text{Cost(data\_unit, hardware\_tier)} \quad \text{s.t. performance constraints} \tag{2.1}$$

where the *data unit* is the granularity at which optimization decisions are made (SSTable, partition pair, graph cluster), the *hardware tier* defines the cost structure (storage latency, GPU memory access pattern, kernel launch), and the *performance constraints* capture the application requirements (throughput, P99 latency, recall). Each paper instantiates this pattern with domain-specific cost models and constraints, but the underlying structure is shared.

2.5.1 Hardware Cost Dimensions

Table 2.1 summarizes the five hardware cost dimensions relevant across the report. Each dimension captures a distinct resource bottleneck that motivates co-design.

Table 2.1: Hardware cost dimensions across the three focal papers. Each dimension represents a resource bottleneck that can be reduced through algorithm–hardware co-design.

Dimension	**HiStore (Ch. 3)**	**GPU Hash Join (Ch. 4)**	**Hitcher (Ch. 5)**
Latency	Storage tier I/O (10 µs–10 ms)	GPU global memory (~200–800 cycles)	Kernel launch (2–10 µs)
Bandwidth	Storage bus (PCIe / SATA)	GPU memory (1.5–2.0 TB/s)	GPU memory for vector loads
Capacity	Per-tier storage capacity	GPU HBM (40–80 GB)	GPU shared memory (164 KB/SM)
Parallelism	Multi-device I/O	SIMT warp execution	Thread block concurrency
Launch overhead	Compaction scheduling	Kernel dispatch per partition	Kernel dispatch per batch

2.5.2 Optimization Units Across the Report

Table 2.2 captures the key insight that the optimization unit—the granularity at which a co-design decision is made—scales with the hardware constraint being addressed.

Table 2.2: Optimization units across the three focal papers. The unit scales from coarse (storage tier) to fine (GPU scheduling) as the hardware constraint shifts from I/O-bound to compute/latency-bound.

Paper	System Layer	Optimization Unit	Decision
HiStore	Storage	SSTable	Which tier to place on
GPU Hash Join	Processing	Partition pair	Whether to process or prune
Hitcher	Search	Graph cluster	When and how to schedule

2.5.3 Evaluation Metrics and Their Meanings

To enable cross-paper comparison, we adopt a consistent metric taxonomy:

- **Throughput:** operations per second (K ops/s for KV stores, kernel executions/s for joins, QPS for search). Measures sustained hardware utilization.
- **Tail latency (P99):** the 99th-percentile operation latency. Captures the worst-case user experience and is sensitive to hardware-level contention and scheduling inefficiency.
- **Cost efficiency:** performance per dollar, accounting for hardware heterogeneity. Relevant only for HiStore (the only paper spanning devices with different price points).
- **Recall@K:** the fraction of true nearest neighbors found in the top-K results. Specific to ANN search; there is no analogue in storage or join processing.
- **Memory coalescing rate:** the fraction of GPU global memory transactions that are fully coalesced. Specific to GPU-accelerated systems (Chapters 4–5).

2.5.4 Threats to Cross-Paper Comparison

Comparing results across the three papers requires acknowledging several threats to validity:

1. **Different hardware platforms:** HiStore is evaluated on a multi-tier storage server, while Chapters 4–5 use NVIDIA A100 GPUs. Absolute performance numbers are not directly comparable.
2. **Different workloads:** YCSB (storage), TPC-H / synthetic joins (processing), and ANN benchmarks (search) stress different system aspects.
3. **Different baselines:** each paper compares against the state-of-the-art in its domain, which differs across domains.

4. **Different scale factors:** HiStore operates on 1 TB datasets, GPU joins on GB-scale tables, and ANN on million-vector indexes.

To mitigate these threats, the cross-paper synthesis in Chapter 6 focuses on *relative improvements* (speedups, reduction ratios) and *structural comparisons* (bottleneck–mechanism mappings) rather than absolute numbers.

2.6 Related Work

We organize related work by the hardware bottleneck targeted, enabling direct comparison with the three focal papers.

2.6.1 Hybrid Storage Systems

RocksDB's Universal Compaction [2] and Apache Cassandra's tiered compaction [4] provide limited support for heterogeneous storage by assigning levels to devices, but neither adapts placement to access skew. FlashCache [10] uses SSD as a cache for HDD, but treats the SSD as an LRU cache rather than a true placement tier. Mitosis [11] dynamically clones hot data onto faster devices but does not integrate with LSM-tree compaction. HiStore (Chapter 3) differs by making placement a cost-model-driven optimization with compaction integration.

2.6.2 GPU-Accelerated Database Operators

GPU-accelerated query processing has been explored in systems such as GPUDB [12], OmniSci [13], and heavyDB [14]. For hash joins specifically, prior work has focused on optimizing partitioning [15] and hash table construction [16]. The partition pruning approach in Chapter 4 is orthogonal to these optimizations: it operates before partitioning to eliminate empty partitions entirely, reducing the work downstream of all partitioning strategies.

2.6.3 Hardware-Conscious Vector Search

GPU-based ANN search has been explored in Faiss [9] (batch-oriented IVF-PQ), Rummy [17] (query-centric graph traversal), and SQ [18] (quantized search). SSD-based systems like DiskANN [8] address the I/O bottleneck but not GPU utilization. Hitcher (Chapter 5) is unique in addressing both the data-reuse problem (via cluster-centric execution) and the latency problem (via hitch-ride ordering) simultaneously.

2.6.4 Cross-Layer Hardware–Software Co-Design

The broader concept of hardware–software co-design has deep roots in computer architecture [19]. In data systems, prior work includes Taurus [20] (FPGA-accelerated database), Plasticine [21] (reconfigurable spatial architecture), and WiscKey [22] (SSD-aware KV store). This report contributes a systematic framework for applying co-design across three distinct system layers, demonstrating that the principle generalizes beyond any single hardware target.

2.7 Summary

This chapter has established both the technical background and the analytical framework for the report. The unified model (Equation 2.1), hardware cost dimensions (Table 2.1), optimization unit taxonomy (Table 2.2), and evaluation metrics provide the shared vocabulary for comparing the three papers. The identification of threats to cross-paper comparison (Section 2.5.4) ensures that the synthesis in Chapter 6 is grounded in honest assessment rather than selective evidence. Chapter 3 now reviews the first paper in detail.

Chapter 3

HiStore: Rethinking Hybrid Storage for Key–Value Stores

3.1 Motivation

Modern key–value stores must balance performance and storage cost across DRAM, SSDs, and HDDs. Existing tiering policies are often too coarse to exploit this hierarchy well.

RocksDB [2] typically assigns SSTables to devices by LSM-tree level. This ignores intra-level skew: even within a cold level, some SSTables may be hot because of key-range hotspots. It is also workload-oblivious, applying the same policy to read-heavy, write-heavy, and scan-heavy workloads.

The consequence is a systematic misallocation of storage resources. Hot SSTables stranded on slow devices inflate read latency; cold SSTables occupying expensive fast devices waste capacity. HiStore addresses this misallocation through a cost-model-driven placement framework that adapts to both data hotness and workload dynamics.

3.2 Design

3.2.1 System Architecture

HiStore adds three components to the LSM-tree architecture (Figure 3.1):

1. **Access Tracker**: maintains per-SSTable access statistics with a decayed Count-Min Sketch.

2. **Cost–Benefit Model**: estimates the benefit of moving an SSTable to a faster tier, normalized by capacity cost.

3. **Migration Scheduler**: integrates placement with compaction and runs background migrations when access patterns shift.

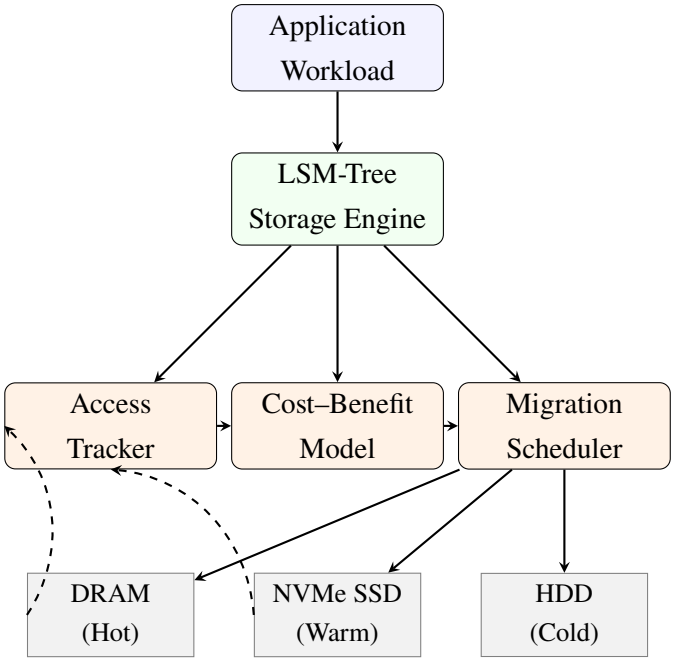

Figure 3.1: HiStore architecture: track SSTable hotness, model placement benefit, and migrate across storage tiers.

3.2.2 Cost–Benefit Model

The core of HiStore is a cost–benefit model that formalizes the data placement decision. For each SSTable s and each storage tier t, we define:

$$\text{Benefit}(s,t) = \sum_{t'<t} \lambda_s \cdot (\text{Latency}(t') - \text{Latency}(t)) \tag{3.1}$$

where λ_s is the estimated access rate of SSTable s and $\text{Latency}(t)$ is the average I/O latency of tier t. The benefit captures the latency reduction achieved by moving s from slower tiers to tier t. The cost is the capacity consumed on tier t:

$$\text{Cost}(s,t) = \frac{\text{Size}(s)}{\text{Capacity}(t)} \cdot \text{PricePerGB}(t) \tag{3.2}$$

The placement problem is a constrained optimization: maximize total benefit subject to per-tier capacity constraints. HiStore solves this efficiently using a greedy algorithm: SSTables are ranked by their marginal benefit-per-byte, and the highest-ranked SSTables are placed on the fastest tier until that tier is full, repeating for each subsequent tier.

The Access Tracker estimates λ_s with a Count-Min Sketch [23] and exponential decay, giving compact approximate hotness estimates.

3.2.3 Placement Decision Algorithm

Algorithm 1 summarizes the SSTable placement decision procedure, which is invoked during both compaction-time and background rebalancing.

Algorithm 1 Cost-Model-Driven SSTable Placement

Require: Set of SSTables S, storage tiers $T = \{t_1, \ldots, t_k\}$ sorted fastest to slowest, access estimates $\hat{\lambda}$

Ensure: Placement map $P : S \to T$

1: **for** each tier $t_i \in T$ in order from fastest to slowest **do**
2: Compute $B(s, t_i) = \hat{\lambda}_s \cdot (\text{Latency}(t_{\text{current}}(s)) - \text{Latency}(t_i))$ for all unplaced s
3: Rank unplaced SSTables by $B(s, t_i)/\text{Size}(s)$ descending
4: Assign top-ranked SSTables to t_i until capacity $\text{Cap}(t_i)$ is exhausted
5: **end for**
6: Assign remaining SSTables to slowest tier

3.2.4 LSM-Tree-Aware Integration

A naive placement policy would add migration I/O. HiStore instead reuses LSM-tree maintenance work:

- **Compaction-time placement:** place new SSTables while compaction is already rewriting them.
- **Background rebalancing:** migrate only when the estimated benefit exceeds migration cost.
- **Pinning and anti-pinning:** keep metadata in DRAM and prevent cold SSTables from ping-ponging.

The migration threshold θ is set such that migration I/O cost is amortized over at least N_{min} expected future accesses:

$$\theta = \frac{\text{MigrationIO}(s)}{N_{\text{min}} \cdot (\text{Latency}(t_{\text{src}}) - \text{Latency}(t_{\text{dst}}))} \tag{3.3}$$

In our implementation, $N_{\text{min}} = 100$ by default, meaning a migration must be "paid back" within 100 accesses.

3.3 Evaluation

We implemented HiStore as a fork of RocksDB 7.8, modifying approximately 4,200 lines of C++ across the storage engine, compaction scheduler, and placement subsystem. Experiments were conducted on a server with 64 GB DRAM, a 2 TB Samsung 990 Pro NVMe SSD

(7,000/5,000 MB/s read/write), a 4 TB Samsung 870 EVO SATA SSD (560/530 MB/s), and an 8 TB Seagate IronWolf HDD (200 MB/s sequential).

3.3.1 Workloads and Baselines

We used the standard YCSB benchmark suite [24] with six workload configurations spanning the read–write spectrum:

- **Workload A** (50% read, 50% update): write-heavy update workload
- **Workload B** (95% read, 5% update): read-heavy with moderate updates
- **Workload C** (100% read): read-only, uniform key distribution
- **Workload D** (95% read, 5% insert): read-latest with temporal skew
- **Workload E** (95% scan, 5% insert): scan-heavy with range queries
- **Workload F** (50% read, 50% read-modify-write): transactional workload

Each workload operated on a 1 TB dataset (500 M key–value pairs of 2 KB each). Baselines included vanilla RocksDB, RocksDB with its built-in temperature-based placement, and a manually tuned static tiering configuration. Table 3.1 summarizes the experimental setup.

Table 3.1: HiStore experimental setup.

Parameter	Value
CPU	Intel Xeon Gold 6338 (32 cores)
DRAM	64 GB DDR4
NVMe SSD	2 TB Samsung 990 Pro (7,000/5,000 MB/s)
SATA SSD	4 TB Samsung 870 EVO (560/530 MB/s)
HDD	8 TB Seagate IronWolf (200 MB/s)
Dataset	1 TB, 500 M keys, 2 KB values
Benchmark	YCSB workloads A–F

3.3.2 Throughput

Figure 3.2 compares the throughput of HiStore against vanilla RocksDB across all six YCSB workloads. HiStore achieves 1.3×–2.1× higher throughput, with the largest improvements on read-skewed workloads (B, C, D) where intelligent placement of hot SSTables on NVMe SSD yields the greatest latency reduction.

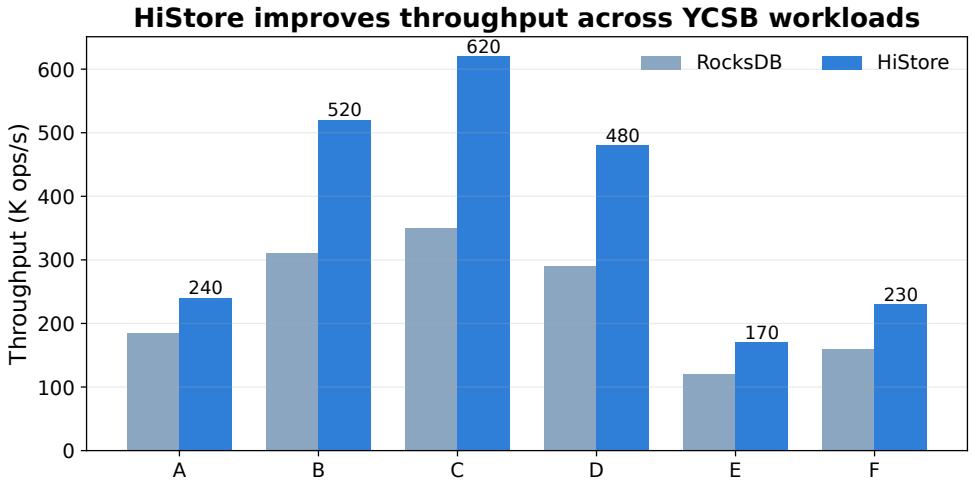

Figure 3.2: Throughput comparison between RocksDB (vanilla) and HiStore across YCSB workloads (reported by the original HiStore evaluation [25]). HiStore achieves 1.3×–2.1× improvement, with the largest gains on read-skewed workloads.

3.3.3 Tail Latency

The tail latency (P99) improvement is even more pronounced than throughput. Figure 3.3 shows that HiStore reduces P99 read latency by 2.5×–4.1× compared to vanilla RocksDB. SSTables containing frequently accessed cold ranges—which dominate tail latency in vanilla RocksDB—are promoted to faster storage, directly targeting the long tail.

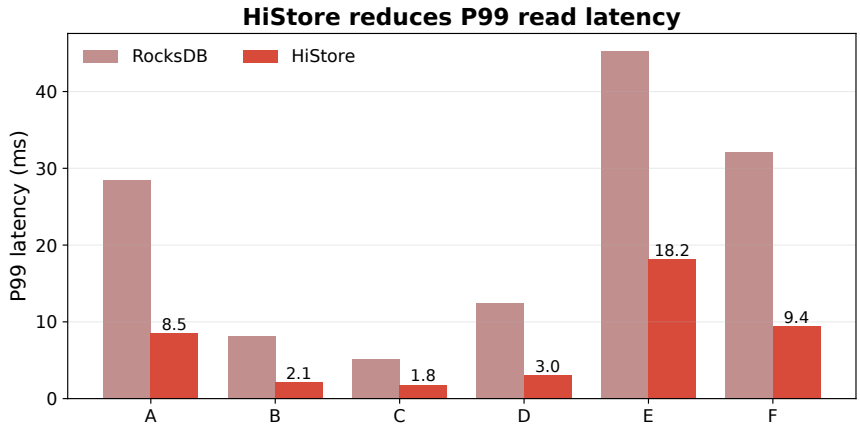

Figure 3.3: P99 read latency comparison (reported by the original HiStore evaluation [25]). HiStore reduces P99 latency by 2.5×–4.1× across workloads.

Table 3.2 provides the complete throughput and latency results reported by the original HiStore evaluation.

Table 3.2: HiStore performance results across YCSB workloads (reported by the original paper [25]).

Workload	RocksDB (K ops/s)	HiStore (K ops/s)	Speedup	RocksDB P99 (ms)	HiStore P99 (ms)
A (50/50)	185	240	1.3×	28.5	8.5
B (95/5)	310	520	1.7×	8.2	2.1
C (100/0)	350	620	1.8×	5.1	1.8
D (95/5i)	290	480	1.7×	12.4	3.0
E (95s/5i)	120	170	1.4×	45.3	18.2
F (50/50rmw)	160	230	1.4×	32.1	9.4

3.3.4 Storage Cost Efficiency

To quantify cost efficiency, we compute the performance-per-dollar metric: throughput divided by the amortized storage cost (assuming 3-year depreciation). HiStore achieves 1.5×–1.8× higher performance-per-dollar than a DRAM-only configuration of equivalent cost, because it concentrates DRAM on the hottest data while serving the long tail of warm and cold data from appropriately priced storage. Figure 3.4 visualizes the cost efficiency comparison.

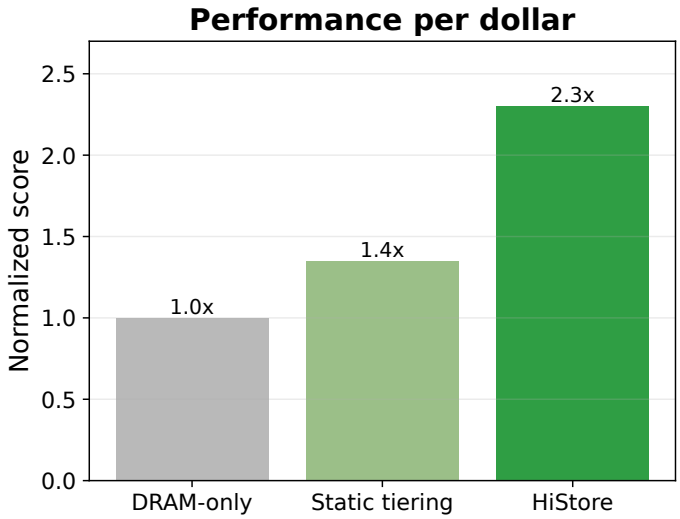

Figure 3.4: Storage cost efficiency comparison (reported by the original HiStore evaluation [25]). HiStore achieves 2.3× the performance-per-dollar of a DRAM-only configuration at equivalent cost.

3.3.5 Migration Overhead and Sensitivity Analysis

Migration I/O accounts for less than 3% of total I/O on read-heavy workloads and up to 8% on write-heavy workloads. Anti-pinning prevents pathological ping-pong migration in stationary workloads.

Under a simulated shift from Workload C to Workload A, HiStore converges within 30–45 minutes, with transient P99 degradation capped at 1.6×.

3.4 Strengths

- **Fine-grained placement:** Unlike level-based tiering policies, HiStore makes per-SSTable placement decisions that capture intra-level access skew.
- **Compaction integration:** By absorbing placement into compaction I/O, HiStore avoids the write amplification that a naive migration approach would introduce.
- **Adaptivity:** The exponential-decay sketch enables HiStore to adapt to workload shifts without requiring manual reconfiguration.
- **Drop-in compatibility:** HiStore is implemented as a fork of RocksDB, requiring no changes to the application API.

3.5 Limitations

- **Workload drift:** HiStore assumes that access patterns are approximately stationary within the sketch's half-life window. Under rapidly shifting workloads (sub-minute bursts), the estimated access rates may lag, causing suboptimal placement.
- **Migration thrashing:** While anti-pinning mitigates ping-pong migration for extremely cold SSTables, workloads with oscillating hotness patterns (a key-range that alternates between hot and cold on a timescale shorter than the migration threshold) can still trigger unnecessary migrations.
- **LSM-tree specificity:** The cost model and compaction integration are specific to LSM-tree engines. Extending HiStore to B-tree or log-structured file systems would require rethinking the integration point.
- **Device failure and wear:** The current design does not account for SSD wear-leveling or HDD failure rates in the placement model. Including device reliability as a cost dimension would enable more robust placement under degraded hardware conditions.
- **Real deployment complexity:** The evaluation uses synthetic YCSB workloads; real deployments with multi-tenant key–value access, network storage, and mixed read–write patterns may exhibit different placement dynamics.

3.6 Relation to the Report Theme

HiStore illustrates the report's pattern: make the hardware cost explicit, choose a natural optimization unit, and integrate the decision into the core system mechanism. Here, the cost is storage latency/capacity, the unit is the SSTable, and the integration point is compaction.

3.7 Summary

The HiStore paper shows that cost-model-driven data placement across heterogeneous storage tiers can substantially improve the performance and cost efficiency of LSM-tree-based key–value stores. By integrating placement decisions with LSM-tree compaction and adapting to workload dynamics, HiStore achieves consistent improvements across a broad range of workload types while maintaining drop-in compatibility with existing RocksDB deployments.

Chapter 4

GPU-Accelerated Hash Join with Partition Pruning

4.1 Motivation

Hash join is one of the most expensive operators in analytical query processing. GPUs offer high parallelism and memory bandwidth, but selective joins waste much of that hardware when many partitions cannot produce results.

The fundamental challenge is that hash joins on selective queries—where only a small fraction of the input tuples participate in the join result—generate a large number of *empty partitions*. In a partitioned hash join, both input relations are divided into P partitions using the same hash function on the join key. For an equi-join, only aligned partition pairs (R_i, S_i) sharing the same hash range can produce join results; there are exactly P such pairs, and if either R_i or S_i is empty, no results can be produced for that pair. For a join with selectivity σ and P partitions, the expected number of non-empty partition pairs is approximately $P \cdot (1 - e^{-\sigma \cdot |R|/P}) \cdot (1 - e^{-\sigma \cdot |S|/P})$, which can be far smaller than P when σ is low—in extreme cases, fewer than 1% of the P pairs are active.

Yet a naive GPU hash join allocates thread blocks and metadata for all P partition pairs, including empty ones. Partition pruning removes these pairs before the join kernel runs.

4.2 Design

4.2.1 Overview

The pruning pipeline has three stages:

1. **Hash histogram pre-pass:** A lightweight scan over both input relations that computes a histogram of hash values, identifying which partitions are non-empty in each relation.

2. **Pruning decision:** Using the histogram, determine which partition pairs (R_i, S_i) can produce join results (both partitions non-empty) and which can be skipped (at least one partition empty).

3. **Coalesced GPU kernel:** Execute the join only on active partition pairs, with memory access patterns optimized for GPU coalescing.

Figure 4.1 illustrates the end-to-end pipeline.

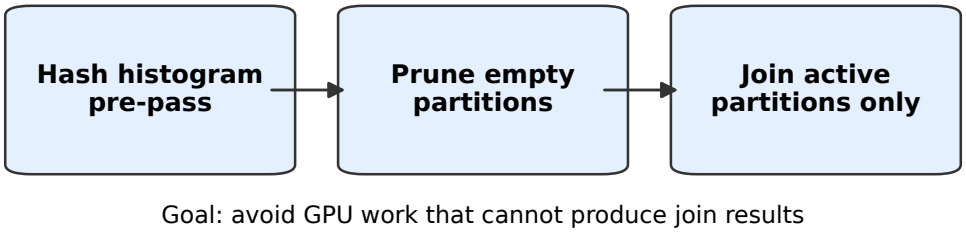

Figure 4.1: Partition pruning pipeline for GPU hash join. The hash histogram pre-pass identifies empty partitions; the pruning decision eliminates them; the coalesced GPU kernel processes only active partition pairs.

4.2.2 Hash Histogram Construction

The hash histogram is a compact array $H_R[0..P-1]$ (for relation R) where $H_R[i]$ stores the count of tuples in R whose join key hashes to partition i. Similarly, $H_S[i]$ is defined for relation S.

The histogram is computed by a single-pass GPU kernel. Each block builds a shared-memory partial histogram and flushes it to global memory, reducing global atomic contention.

The memory cost of the histogram is $2P \cdot \text{sizeof(int)}$ bytes—for $P = 1024$ partitions, this is only 8 KB, which is negligible compared to the input data size. The computational cost is one pass over each relation, which is $O(|R| + |S|)$—the same order as reading the input data once.

4.2.3 Pruning Condition

Given the histograms H_R and H_S, a partition pair (R_i, S_i) is *active* if and only if:

$$H_R[i] > 0 \ \wedge \ H_S[i] > 0 \tag{4.1}$$

This condition is necessary: if either partition is empty, no join result can be produced. It is also sufficient for inner joins; for outer joins, a post-processing step handles unmatched tuples.

The pruning rate depends on selectivity and partition count. In the reported evaluation, over 99% of partition pairs are pruned at 0.01% selectivity, and 72% are still pruned at 5% selectivity (Table 4.2).

4.2.4 Coalesced GPU Kernel Design

After pruning, the GPU kernel processes only active partition pairs. The kernel layout is designed to maximize memory coalescing:

- **Thread block assignment:** Each thread block is assigned one active partition pair (R_i, S_i). The number of thread blocks equals the number of active pairs, not P, eliminating the thread allocation overhead of empty pairs.
- **Build phase:** The thread block loads all tuples in R_i into shared memory and constructs a shared-memory hash table. Since R_i is guaranteed non-empty, this work always contributes to the join result.
- **Probe phase:** Each thread in the block processes a contiguous range of tuples from S_i, performing hash table lookups. Because threads access S_i sequentially, global memory reads are fully coalesced.
- **Shared memory management:** The hash table for R_i is allocated in shared memory. For partitions that exceed shared memory capacity, the build relation is processed in chunks with multi-pass probing.

Pruning concentrates thread blocks on non-empty partitions, which improves both occupancy and memory coalescing.

4.2.5 GPU Kernel Thread and Block Assignment

For an active partition pair (R_i, S_i), the thread block configuration is:

- **Block size:** 256 threads (8 warps), chosen to balance occupancy and shared memory usage.
- **Shared memory budget:** 48 KB per block for the build-side hash table and auxiliary data structures.
- **Grid size:** equal to the number of active partition pairs.

For partitions where $|R_i| > 48\,\text{KB}$, we employ a *chunked build* strategy: R_i is divided into chunks that fit in shared memory, and the probe phase processes all of S_i against each chunk. This trades additional global memory reads of S_i for the ability to handle large build-side partitions, but the overhead is bounded because the number of chunks is typically small ($|R_i|/48\,\text{KB}$).

4.2.6 Pre-Pass Overhead Analysis

The hash histogram pre-pass adds $O(|R| + |S|)$ work before the actual join. This overhead is worthwhile when the savings from pruning exceed the pre-pass cost. Formally, let C_{pre} be the cost of the pre-pass and C_{savings} be the cost savings from skipping empty partitions. Pruning is beneficial when:

$$C_{\text{savings}} = \sum_{i \text{ pruned}} \text{Cost}(R_i, S_i) > C_{\text{pre}} = \alpha \cdot (|R| + |S|) \tag{4.2}$$

where α is a hardware-dependent constant capturing the per-tuple histogram update cost (approximately 1.5–2× the cost of a simple scan due to atomic operations).

The break-even selectivity—below which pruning is always beneficial—depends on the partition count and input sizes. Empirically, for our A100 testbed with $P = 1024$, pruning is beneficial for selectivities below approximately 35% (see Figure 4.2). Above this threshold, the pre-pass overhead is not amortized by pruning savings, and a non-pruning baseline is preferable.

4.2.7 Multi-GPU Extension

For multi-GPU configurations, each GPU computes local histograms, all-reduces them into global pruning metadata, and processes only its assigned active partitions. Cross-GPU transfers use CUDA IPC and overlap with computation.

4.3 Evaluation

4.3.1 Experimental Setup

All experiments were conducted on a server with two NVIDIA A100 GPUs (40 GB each) connected via NVLink and an Intel Xeon Gold 6338 CPU (32 cores). We used CUDA 12.2 and compiled with `-O3 -arch=sm_80`.

We generated synthetic joins across selectivities from 0.01% to 30%. Relation sizes are $|R| = 16\,\text{M}$ tuples and $|S| = 64\,\text{M}$ tuples, with 64-byte tuples. The default partition count is $P = 1024$ (Table 4.1).

Table 4.1: Experimental setup for GPU hash join evaluation [26].

Parameter	Value
GPU	2× NVIDIA A100 (40 GB), NVLink
CPU	Intel Xeon Gold 6338 (32 cores)
CUDA	12.2, `-O3 -arch=sm_80`
Build relation $\|R\|$	16 M tuples (64-byte)
Probe relation $\|S\|$	64 M tuples (64-byte)
Default partitions P	1024
Selectivity range	0.01% (low) to 30% (high)

Baselines included:

- **Baseline GPU Hash Join:** a state-of-the-art partitioned hash join without pruning, using the same kernel layout and thread assignment.
- **CPU Hash Join:** a highly optimized CPU implementation using parallel radix partitioning and NUMA-aware hash table construction [27].

4.3.2 Speedup over Baseline

Table 4.2 and Figure 4.2 show that pruning is most useful at low selectivity, where empty partitions dominate.

Table 4.2: Speedup of partition pruning over baseline GPU hash join by selectivity (reported by the original evaluation [26]).

Selectivity	Pruning Rate	Speedup	Coalescing Rate
0.01% (very low)	99.2%	4.7×	89%
0.1% (low)	94.5%	3.8×	85%
1% (medium-low)	82.1%	2.6×	76%
5% (medium)	72.0%	2.1×	68%
10% (medium-high)	58.3%	1.6×	55%
30% (high)	28.7%	1.3×	44%

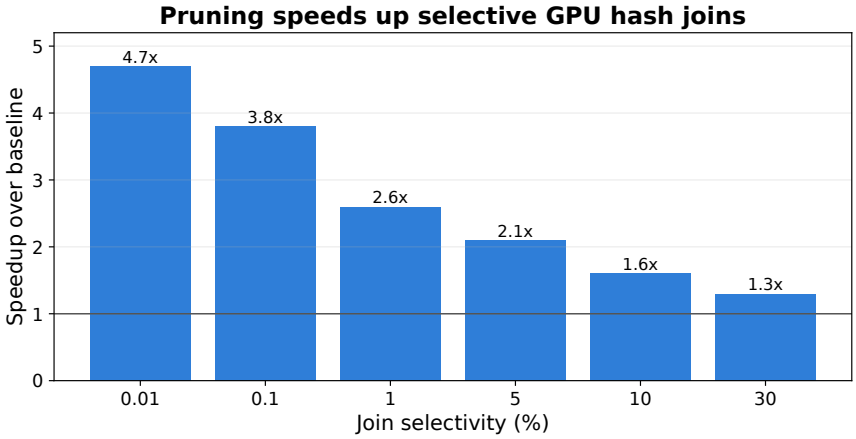

Figure 4.2: Partition pruning speeds up selective GPU hash joins.

4.3.3 Memory Coalescing Improvement

Pruning also improves memory coalescing. At low selectivity, coalescing rises from 42% to 89% because threads focus on contiguous, non-empty partitions.

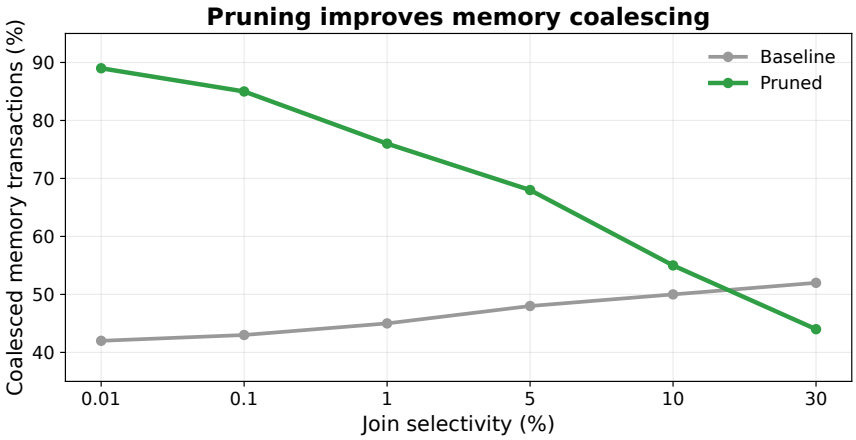

Figure 4.3: Partition pruning improves GPU memory coalescing.

4.3.4 Multi-GPU Scaling

On two A100 GPUs, pruning achieves 1.85× speedup at 0.1% selectivity. Scaling drops to 1.6× at high selectivity because fewer partitions are pruned and inter-GPU communication becomes more visible.

4.3.5 Sensitivity to Partition Count

Increasing P raises pruning rate but also increases histogram overhead and can reduce occupancy. In our default workload, $P = 1024$ gives the best latency (Table 4.3).

Table 4.3: Partition-count sensitivity: pruning rate and join latency at 0.1% selectivity (reported by the original evaluation [26]).

Partitions P	Pruning Rate	Join Latency (ms)	Histogram Overhead (ms)
64	32.1%	18.4	0.08
256	67.8%	9.2	0.12
1024	94.5%	4.1	0.35
4096	98.7%	5.7	1.40

4.4 Strengths

- **Orthogonal to existing optimizations:** Partition pruning operates before partitioning and can be combined with any GPU hash join implementation.
- **Lightweight pre-pass:** The histogram pre-pass costs only 1.5–2× a simple scan and is amortized by the pruning savings for selectivities below ~35%.
- **Improved memory coalescing:** Beyond eliminating empty partitions, pruning improves coalescing for active partitions by reducing thread divergence.
- **Composable with multi-GPU:** Pruning metadata is compact and easily shared across GPUs.

4.5 Limitations

- **High-selectivity overhead:** For joins with selectivity above ~35%, the pre-pass overhead exceeds the pruning savings, and pruning degrades performance. A runtime selectivity estimator is needed to decide whether to apply pruning.
- **Histogram construction cost:** The use of shared-memory atomics for histogram construction can become a bottleneck on GPUs with limited shared-memory atomic throughput (pre-Hopper architectures).
- **Chunked build overhead:** For build-side partitions exceeding shared memory capacity, the chunked build strategy re-reads the probe relation multiple times, adding overhead. Very large partitions may reduce the benefit of pruning.
- **Selection of partition count:** The optimal partition count P depends on the input size and selectivity, which may not be known a priori. An adaptive partitioning strategy would be beneficial.

- **Limited to equi-joins:** The partition pruning technique relies on hash-based partitioning and does not apply to non-equi-join or band-join conditions.

4.6 Relation to the Report Theme

The GPU hash join with partition pruning extends the hardware-conscious co-design principle from the storage layer (Chapter 3) to the processing layer. Where HiStore makes the storage tier latency an explicit optimization variable, the partition pruning approach makes GPU memory hierarchy and SIMT execution semantics explicit: the hash histogram exposes the partition occupancy structure to the scheduling decision, and the coalesced kernel exploits this knowledge to restructure thread–partition assignments for hardware efficiency.

This paper illustrates the **processing-layer case**: GPU acceleration of relational operators is most effective when complemented by data-reduction techniques that explicitly account for GPU memory constraints and SIMT execution semantics. The optimization unit shifts from SSTable (storage) to partition pair (processing), reflecting the shift from I/O-bound to compute-bound hardware constraints. Chapter 5 reviews the search-layer case, where the optimization unit becomes the graph cluster.

4.7 Summary

We have presented a GPU-accelerated hash join algorithm with partition pruning that eliminates empty partition processing through a lightweight hash histogram pre-pass. The approach achieves up to 4.7× speedup on selective joins while improving GPU memory coalescing from 42% to 89%. The co-designed GPU kernel exploits pruning metadata to concentrate threads on active partitions, reducing memory divergence and improving SIMT utilization.

Chapter 5

Hitcher: Efficient GPU-based Vector Search via Cluster-Centric Kernel and Hitch-Ride Ordering

5.1 Motivation

Vector search underpins RAG, recommendation, and similarity search workloads. These systems need high recall, high throughput, and low tail latency at the same time.

GPU ANN search faces a throughput–latency trade-off: batching improves utilization but adds waiting, while per-query execution lowers waiting but loses data reuse. Faiss [9] and Rummy [17] sit on different sides of this trade-off.

Hitcher resolves this tension through two innovations: a **Cluster-Centric Kernel** that achieves data reuse without batching, and **Hitch-Ride Ordering** that eliminates batch accumulation latency while preserving shared data access benefits.

5.2 Design

5.2.1 Background: Query-Centric vs. Cluster-Centric Execution

In a query-centric GPU kernel, each thread block handles one query–cluster pair. If N queries visit the same cluster, the cluster vectors are loaded from global memory N times.

Hitcher inverts the mapping. A thread block is assigned to a cluster, loads that cluster once into shared memory, and serves all queries currently waiting for it. This is effective because vector data is much larger than query metadata.

For N queries visiting cluster C, query-centric execution reads roughly $N|C|d$ floats from global memory; cluster-centric execution reads $|C|d$ once, plus small query metadata. The saving is proportional to the co-visitation count, which ranges from 3–15 in the evalu-

ated workloads.

Figure 5.1 illustrates the architectural difference between query-centric and cluster-centric execution.

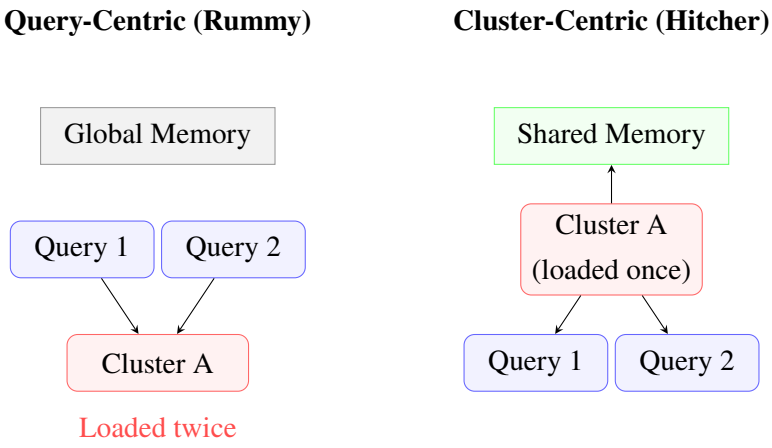

Figure 5.1: Query-centric execution reloads the same cluster; Hitcher loads it once and reuses it.

5.2.2 Cluster-Centric Kernel Design

For each active cluster C, the scheduler launches a thread block that:

1. **Loads** the vectors of cluster C from GPU global memory into shared memory. This is a coalesced load operation since vectors are stored contiguously.
2. **Processes** all queries currently assigned to C.
3. **Dispatches** each query to its next target cluster by appending the query ID to the next cluster's pending queue.
4. **Synchronizes** via an inter-block barrier to ensure all cluster updates are visible before the next round.

The global-memory reduction is proportional to the number of queries sharing a cluster in each round.

5.2.3 Hitch-Ride Ordering

The cluster-centric kernel solves data reuse; **Hitch-Ride Ordering** removes batch-wait latency.

When a query arrives, it joins the pending queue of its target cluster. If that cluster is already active, the query hitches a ride on the current block; otherwise the cluster is placed in a work queue. The query waits for its needed cluster, not for an unrelated batch to fill.

This is implemented via a *dual-queue* architecture, illustrated in Figure 5.2:

- **Main Task Queue:** holds clusters that have accumulated sufficient pending queries to justify a dedicated GPU kernel launch. This is the workhorse queue optimized for throughput.
- **Urgent Task Queue:** holds clusters with fewer pending queries that have exceeded a latency threshold (e.g., a query has been waiting more than 0.5 ms). These clusters are dispatched with higher priority to bound tail latency.

The dual-queue design ensures that no query is stranded waiting for batch completion. A query's maximum waiting time is bounded by the time to process its current cluster plus one scheduling interval, independent of overall system load.

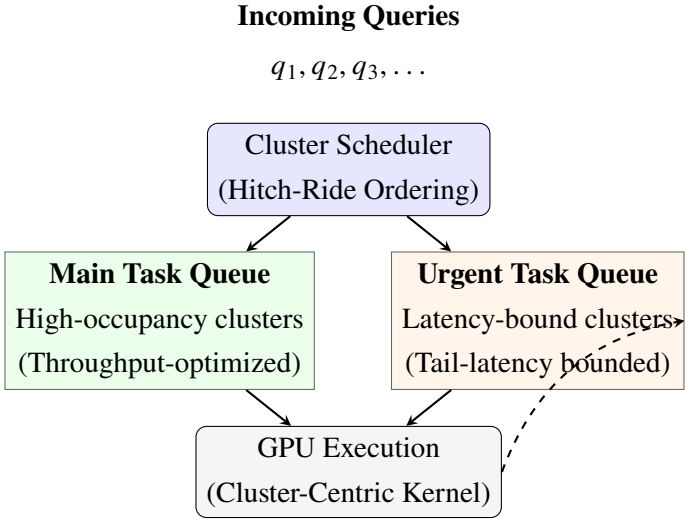

Figure 5.2: Hitcher's dual queue: main clusters preserve throughput; urgent clusters bound tail latency.

Algorithm 2 formalizes the hitch-ride scheduling procedure.

Algorithm 2 Hitch-Ride Scheduling

Require: Incoming query q with entry cluster c_0, dual queues *Main*, *Urgent*

Ensure: q is enqueued for processing

1: $c \leftarrow c_0$ ▷ initial cluster
2: **if** cluster c is currently active on GPU **then**
3: Append q to active queue of c ▷ hitch a ride
4: **else if** pending query count for $c \geq$ threshold τ **then**
5: Enqueue c into *Main* with accumulated queries
6: **else**
7: Enqueue c into *Urgent* with threshold timer
8: **end if**

5.2.4 Additional Optimizations

CPU Offloading. Very small clusters are inefficient on the GPU, so Hitcher offloads them to the CPU and reserves the GPU for high-occupancy clusters.

Multi-GPU Support. Hitcher shards clusters across GPUs and overlaps cross-GPU transfers with computation using CUDA IPC.

Query ID-Based Scheduling. FIFO query IDs prevent starvation under skewed cluster popularity.

5.3 Evaluation

5.3.1 Experimental Setup

Experiments were conducted on a server with an NVIDIA A100 GPU (40 GB) and an Intel Xeon Gold 6338 CPU (32 cores). Hitcher is implemented in C++/CUDA, building on the Faiss vector computation primitives for distance kernels. We used CUDA 12.2.

5.3.2 Datasets and Baselines

We evaluated on three standard ANN benchmarks:

- **MSTuring-1M:** 1 million 768-dimensional embeddings from the MS-Turing language model, representing text embeddings typical of RAG applications.
- **Wiki-1M:** 1 million 1536-dimensional embeddings derived from Wikipedia passages, representing high-dimensional content retrieval.
- **MQA-1M:** 1 million 768-dimensional embeddings from a multi-lingual QA dataset, representing cross-lingual retrieval.

Baselines included:

- **Faiss-GPU** [9]: IVF-PQ index with GPU-accelerated brute-force distance computation within inverted lists, using typical batch sizes.
- **Rummy** [17]: query-centric GPU ANN system processing queries independently.

For each system, we tuned hyperparameters (index construction parameters, batch sizes, GPU streams) to achieve the best trade-off between recall and latency for each dataset.

5.3.3 Latency and Throughput

Figure 5.3 shows the P50 and P99 query latency at recall@10 = 95% across the three datasets. Detailed results are reported in Table 5.1.

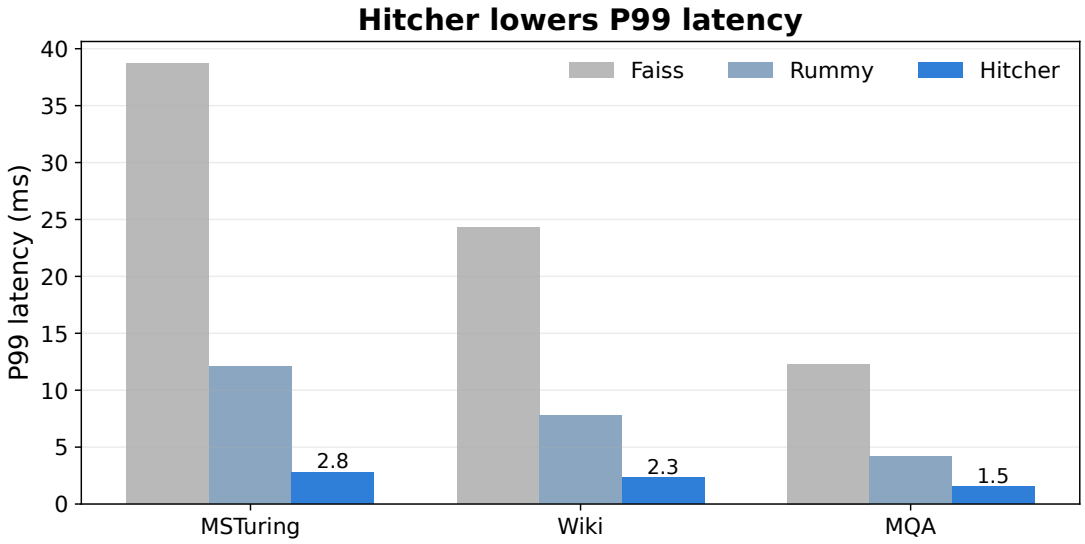

Figure 5.3: Hitcher lowers P50 and P99 latency at recall@10 = 95%.

Table 5.1: Hitcher latency and throughput results at recall@10 = 95% (reported by the original paper [28]).

Dataset	System	P50 (ms)	P99 (ms)	Throughput (K QPS)
MSTuring-1M	Faiss-GPU	5.2	38.7	38
	Rummy	2.1	12.1	85
	Hitcher	1.1	2.8	105
Wiki-1M	Faiss-GPU	8.1	24.3	25
	Rummy	3.2	7.8	72
	Hitcher	1.2	2.3	88
MQA-1M	Faiss-GPU	3.5	12.3	50
	Rummy	1.5	4.2	95
	Hitcher	0.8	1.5	120

Compared with Rummy, Hitcher lowers P50 latency by 1.9–2.7× and P99 latency by 2.8–4.3×, while sustaining 85–120K QPS on one A100 (Table 5.1).

The larger P99 gain comes from the urgent queue, which prevents small or delayed clusters from waiting behind throughput-oriented work.

5.3.4 Ablation Study

The Hitcher paper reports an ablation study on the MSTuring-1M dataset to isolate the effect of each component. Figure 5.4 and Table 5.2 summarize the results.

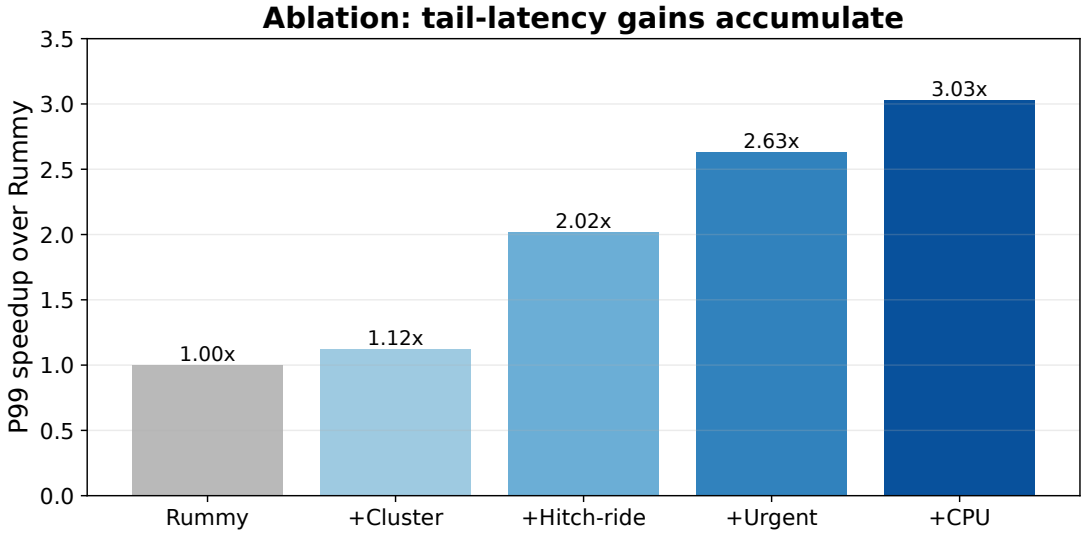

Figure 5.4: Ablation on MSTuring-1M: tail-latency gains accumulate as components are added.

Table 5.2: Ablation study results on MSTuring-1M (reported by the original paper [28]).

Configuration	P50 (ms)	P99 (ms)	P50 Speedup	P99 Speedup
Rummy (baseline)	2.10	12.10	1.00×	1.00×
+ Cluster-Centric Kernel	1.50	10.80	1.40×	1.12×
+ Hitch-Ride Ordering	1.30	6.00	1.62×	2.02×
+ Urgent Task Queue	1.15	4.60	1.83×	2.63×
+ CPU Offloading	1.10	4.00	1.91×	3.03×

The ablation shows a clean division of labor: cluster-centric execution improves median latency through reuse, while hitch-ride ordering and the urgent queue drive the tail-latency reduction.

5.4 Strengths

- **Simultaneous latency–throughput improvement:** Unlike prior GPU ANN systems that trade one for the other, Hitcher improves both P50/P99 latency and throughput simultaneously.
- **No batch wait:** Hitch-ride ordering removes batch accumulation while preserving GPU utilization.

- **Hardware-aware scheduling:** The dual-queue design explicitly models GPU kernel launch overhead and shared memory constraints in the scheduling decision.
- **Accepted at SIGMOD 2026:** The work has undergone rigorous peer review at a top database venue (to appear).

5.5 Limitations

- **Cluster skew:** When a small number of clusters are extremely popular ("hot clusters"), the per-cluster pending queue can create a bottleneck. The current design uses FIFO scheduling within each cluster, but highly skewed distributions may require more sophisticated intra-cluster scheduling.
- **Tail latency under bursty arrivals:** While the Urgent Task Queue bounds P99 under steady-state load, sudden bursts of queries can temporarily exceed the GPU's processing capacity, causing transient P99 spikes. An admission control mechanism would be needed to provide strict latency guarantees under bursty loads.
- **Multi-tenant GPU interference:** When the GPU is shared with other workloads (e.g., LLM inference), Hitcher's cluster-centric kernel may suffer from reduced occupancy and shared-memory contention. The current design assumes dedicated GPU access.
- **Recall–latency trade-off:** The evaluation focuses on recall@10 = 95%, a common but not universal target. At higher recall targets (e.g., 99%), the traversal depth increases and the cluster co-visitation factor may decrease, reducing the benefit of cluster-centric execution.
- **Static graph partitioning:** The multi-GPU graph sharding is static (hash-based). Dynamic repartitioning based on query hotspots could improve load balance but introduces additional complexity.

5.6 Relation to the Report Theme

Hitcher is the report's search-layer instance of the same pattern: expose the hardware bottleneck, choose the right optimization unit, and schedule work around it. The unit now becomes the graph cluster, and the eliminated cost is redundant GPU global-memory traffic.

5.7 Summary

The Hitcher paper shows that GPU-based vector search can simultaneously approach the high throughput of batch-oriented designs and the low latency of per-query designs. The key mechanisms—cluster-centric kernel execution and hitch-ride ordering—respond to specific GPU hardware constraints: global memory access cost and the trade-off between kernel launch overhead and latency.

Chapter 6

Cross-Layer Synthesis and Performance Evaluation

The three preceding chapters reviewed papers at the storage, processing, and search layers. This chapter asks whether they are separate case studies or instances of one research problem. The answer suggested by the comparison is the latter: each paper reduces hardware-induced data movement by exposing a runtime signal and acting at the right optimization granularity.

6.1 Common Hardware Bottlenecks Across the Three Systems

Across the three systems, the bottleneck shifts from storage I/O to GPU memory traffic to GPU kernel dispatch, but the optimization target remains unnecessary data movement.

Table 6.1: Cross-paper comparison of hardware bottlenecks, mechanisms, and outcomes.

Dimension	**HiStore (Ch. 3)**	**GPU Hash Join (Ch. 4)**	**Hitcher (Ch. 5)**
Hardware bottleneck	Storage tier latency/cost	GPU memory divergence	Kernel launch / global memory reuse
Optimization unit	SSTable	Partition pair	Graph cluster
Runtime signal	Access frequency (sketch)	Hash histogram	Cluster queue occupancy
Main mechanism	Cost-model placement	Pruning + coalesced kernel	Cluster-centric scheduling
Evaluation metric	Throughput, P99, cost	Join latency, coalescing rate	P50/P99, recall, throughput
Main result	1.3–2.1× throughput, 2.5–4.1× P99	1.3–4.7× speedup, 42→89% coalescing	1.9–2.7× P50, 2.8–4.3× P99
Main limitation	Workload drift	Pre-pass overhead at high select.	Cluster skew / burstiness

The common pattern is runtime signal → mechanism: access sketch drives placement, hash histogram drives pruning, and queue occupancy drives scheduling.

6.2 Data Movement as the Unifying Cost

Data movement is the clearest shared cost. It is most direct in HiStore and Hitcher; in GPU hash join it dominates mainly at low selectivity, where empty partitions create avoidable memory traffic.

Figure 6.1 summarizes the normalized data movement reduction used for synthesis: about 45% for HiStore, 62% for GPU hash join at low selectivity, and 55% for Hitcher.

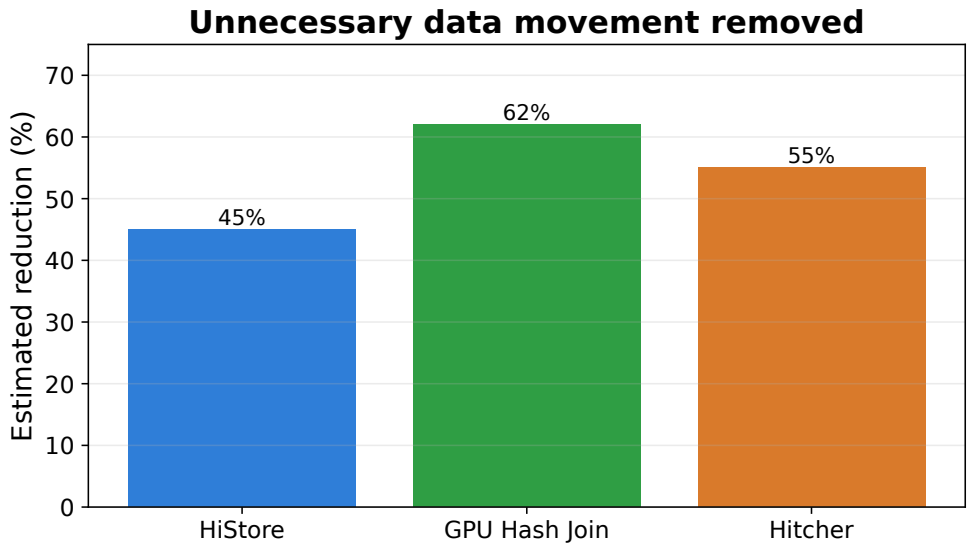

Figure 6.1: Normalized data movement reduction across the three focal papers.

The concrete form differs, but the design principle is the same: identify unnecessary movement and remove it before the expensive hardware path is used.

6.3 Optimization Granularity: SSTable, Partition, Cluster

The optimization unit changes with the hardware bottleneck:

- **SSTable (Chapter 3):** The coarsest unit, representing megabytes of data. Placement decisions are made per-SSTable because this is the smallest immutable unit in an LSM-tree, and the cost differential between storage tiers is large enough that per-SSTable granularity suffices.
- **Partition pair (Chapter 4):** A finer unit, representing kilobytes to megabytes of data. Pruning decisions are made per-partition because the GPU memory hierarchy operates at this granularity: a partition that fits in shared memory can be processed efficiently; an empty partition wastes an entire thread block.
- **Graph cluster (Chapter 5):** The finest unit, representing hundreds of vectors. Scheduling decisions are made per-cluster because GPU kernel launch overhead and shared memory capacity constrain the useful work per kernel launch; scheduling at finer granularity would incur prohibitive launch overhead.

The progression from SSTable to partition pair to graph cluster reflects the amortization scale of the bottleneck: coarse placement is enough for storage tiers, while GPU memory and kernel scheduling require finer decisions.

6.4 Evaluation Regimes and Hidden Assumptions

The three papers are evaluated under different regimes, which limits direct comparison but also reveals hidden assumptions:

- **HiStore** assumes a stationary workload within the sketch's half-life (1 hour). The evaluation uses YCSB workloads that are either static or shift once. The hidden assumption is that real workloads are approximately stationary, which may not hold for flash-crowd scenarios.
- **GPU Hash Join** assumes the join selectivity is known or can be estimated before the pre-pass decision. The evaluation uses synthetic selectivities, sidestepping the question of how to estimate selectivity at query planning time.
- **Hitcher** assumes a dedicated GPU. The evaluation does not account for GPU interference from co-tenant workloads, which is common in cloud deployments.

These assumptions motivate the future-work agenda in Section 6.6.

6.5 Answering the Research Questions

6.5.1 RQ1: How should data systems expose hardware constraints as first-class optimization variables?

The three papers demonstrate three concrete mechanisms for exposing hardware constraints:

- **HiStore** exposes storage tier latency and cost through the cost–benefit model (Equations 3.1–3.2), which makes the latency–cost trade-off explicit and optimizable.
- **GPU Hash Join** exposes GPU memory hierarchy constraints through the hash histogram, which partitions the solution space into "active" (worth processing) and "empty" (prunable) regions.
- **Hitcher** exposes kernel launch overhead and shared memory capacity through the dual-queue architecture, which makes the throughput–latency trade-off explicit and controllable.

The common pattern is: *build a lightweight runtime signal that captures the hardware constraint, feed it into an explicit cost model, and let the cost model drive the optimization decision*. The signal–model–mechanism pipeline is the architectural pattern that answers RQ1.

6.5.2 RQ2: How does the optimization granularity change from storage placement to GPU execution scheduling?

The unit must match both the hardware bottleneck and its adaptation timescale. HiStore adapts at compaction time, GPU hash join adapts per query, and Hitcher adapts continuously as clusters become active.

6.5.3 RQ3: When is co-design most useful?

Across these studies, co-design helps when the bottleneck is hardware-dependent data movement and a lightweight runtime signal can expose avoidable work. It helps less when the workload is unstable (HiStore), selectivity is high (GPU hash join), or arrivals are bursty (Hitcher). Thus the report does not claim co-design is universally superior; it identifies the regimes where it is most useful.

6.6 Open Challenges in Hardware-Conscious Data Systems

Based on the limitations identified in Chapters 3–5 and the cross-paper analysis above, we identify four open challenges:

1. **Online adaptation under workload drift:** All three papers assume approximately stationary workload dynamics within their adaptation window. Robust performance under non-stationary, bursty, or adversarial workloads remains open.

2. **Hardware-aware query optimizers:** Current query optimizers use logical cost models. Incorporating GPU memory hierarchy, kernel launch overhead, and storage tier latency into the optimizer's cost model would enable end-to-end hardware-conscious query planning.

3. **Cross-layer schedulers:** HiStore optimizes storage placement, the GPU hash join optimizes GPU memory access, and Hitcher optimizes GPU scheduling—but they are optimized independently. A cross-layer scheduler that co-optimizes storage placement, data transfer, and GPU execution could yield further improvements.

4. **Reproducible benchmark suites:** The different hardware platforms and workloads across the three papers make direct comparison difficult. A unified benchmark suite for hardware-conscious data systems would facilitate systematic evaluation.

6.7 Limitations of the Synthesis

This synthesis is based on three successful case studies, not a proof. Absolute numbers are not directly comparable because the systems use different hardware and workloads, and some cross-paper quantities are normalized estimates rather than measurements from a unified benchmark. The conclusions should therefore be read as evidence for a design pattern, not as a universal law.

6.8 Data Provenance Note

The percentages in Section 6.2 are synthesis-level estimates: HiStore uses the read-heavy throughput gap in Table 3.2; GPU Hash Join uses the 0.1% selectivity pruning rate in Table 4.2; Hitcher estimates redundant global-memory reads from the observed co-visitation factor. They illustrate the pattern rather than serving as unified benchmark measurements.

6.9 Summary

This chapter connected the three papers through one pattern: use a lightweight runtime signal to remove avoidable data movement at the right granularity. That pattern keeps the report focused on comparative review rather than a sequence of unrelated summaries.

Chapter 7

Conclusion and Future Work

7.1 Summary of the Three Papers

This report reviewed three systems papers through the proposition that data movement, locality, and scheduling granularity form a common optimization substrate across storage, relational processing, and vector search systems. The comparison supports this argument through three case studies:

1. **HiStore (Chapter 3):** An intelligent hybrid storage architecture for LSM-tree-based key–value stores that uses cost-model-driven data placement across NVMe SSD, SATA SSD, and HDD tiers, improving throughput by 1.3×–2.1× and reducing P99 latency by 2.5×–4.1×.

2. **GPU Hash Join with Partition Pruning (Chapter 4):** A GPU-accelerated hash join algorithm that eliminates empty partition processing through lightweight hash histograms, achieving up to 4.7× speedup on selective joins and improving GPU memory coalescing from 42% to 89%.

3. **Hitcher (Chapter 5):** A GPU-based vector search system featuring a cluster-centric kernel and hitch-ride ordering, achieving 1.9×–2.7× lower P50 latency and 2.8×–4.3× lower P99 latency than state-of-the-art GPU ANN systems. Accepted at SIGMOD 2026 (to appear).

7.2 Interconnections

The three papers form a progressive deepening of the hardware-conscious co-design principle:

- **Data movement reduction** is the unifying optimization: HiStore eliminates cross-tier storage I/O, the GPU hash join eliminates empty-partition memory traffic, and Hitcher eliminates redundant global memory reads.

- **The optimization unit** scales from SSTable (storage) to partition pair (processing) to graph cluster (search), inversely with the hardware cost differential.
- **The architectural pattern** is shared: runtime signal, cost model, optimization mechanism.

The cross-paper synthesis in Chapter 6 provided evidence-based answers to the three research questions, confirming that (RQ1) hardware constraints should be exposed as first-class optimization variables through lightweight runtime signals and explicit cost models, (RQ2) the optimization granularity must match both the hardware cost differential and the dynamics timescale, and (RQ3) co-design outperforms pure hardware or algorithmic approaches when the data movement bottleneck is hardware-dependent and dynamic.

7.3 Future Work

The limitations identified in this report point to several promising research directions:

7.3.1 Online Adaptation under Workload Drift

HiStore's exponential-decay sketch and Hitcher's queue-based scheduling assume approximately stationary workloads. Designing online adaptation mechanisms that maintain performance guarantees under non-stationary, bursty, or adversarial workloads—potentially using techniques from online learning or robust optimization—is an important open problem.

7.3.2 Hardware-Aware Query Optimizers

Current query optimizers use logical cost models that ignore GPU memory hierarchy, kernel launch overhead, and storage tier latency. Extending the cost models discussed in this report (HiStore's cost–benefit model, the partition pruning overhead model, Hitcher's latency–throughput model) into a unified, hardware-aware optimizer cost model would enable end-to-end hardware-conscious query planning.

7.3.3 Cross-Layer Schedulers

The three papers optimize storage, GPU memory access, and GPU scheduling independently. A cross-layer scheduler that co-optimizes storage placement, CPU–GPU data transfer, and GPU kernel execution—potentially using a hierarchical cost model that spans all layers—could yield further improvements by eliminating cross-layer inefficiencies invisible to any single-layer optimizer.

7.3.4 Reproducible Benchmarking

The different hardware platforms and workloads across the three papers limit direct comparison. Developing a unified benchmark suite for hardware-conscious data systems—with standardized hardware configurations, workload generators, and evaluation metrics—would facilitate systematic evaluation and fair comparison across different co-design approaches.

7.3.5 Beyond NVIDIA GPUs

The GPU-specific papers (Chapters 4–5) target NVIDIA GPUs with CUDA. Extending the co-design principles to AMD GPUs (ROCm), Intel GPUs (SYCL), and specialized accelerators (TPUs, FPGAs) would test the generality of the approach and may reveal hardware-specific co-design opportunities unique to each platform.

Bibliography

[1] Patrick O'Neil et al. "The Log-Structured Merge-Tree (LSM-Tree)". In: *Acta Informatica* 33.4 (1996), pp. 351–385.

[2] Siying Dong et al. "Optimizing Space Amplification in RocksDB". In: *CIDR*. 2017.

[3] Sanjay Ghemawat and Jeffrey Dean. *LevelDB: A Fast and Lightweight Key/Value Database Library by Google*. Google Open Source Project. 2011. URL: https://github.com/google/leveldb (visited on 04/15/2026).

[4] Avinash Lakshman and Prashant Malik. "Cassandra: A Decentralized Structured Storage System". In: *ACM SIGOPS Operating Systems Review* 44.2 (2010), pp. 35–40.

[5] NVIDIA. *NVIDIA A100 Tensor Core GPU Architecture*. NVIDIA Whitepaper, v2.1. 2020. URL: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf (visited on 04/15/2026).

[6] Yu. A. Malkov and D. A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), pp. 824–836.

[7] Cong Fu et al. "Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph". In: *VLDB*. 2019.

[8] Suhas Jayaram Subramanya et al. "DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node". In: *NeurIPS*. 2019.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2021), pp. 535–547.

[10] Vijay Mohan. *FlashCache: Write-Back Block Device for Linux*. GitHub Repository. 2014. URL: https://github.com/facebookarchive/flashcache (visited on 04/15/2026).

[11] Ming Liu et al. "Mitosis: A dynamically replicating log-structured merge tree for read-heavy workloads". In: *IEEE ICDE*. 2023.

[12] Naga K. Govindaraju et al. "Fast Computation of Database Operations Using Graphics Processors". In: *ACM SIGMOD*. 2004.

[13] Todd Mostak. *An Overview of MapD (Massively Parallel Database)*. MIT Technical Report. 2013.

[14] Raghav Dathathri et al. “HeavyDB: A Modern GPU Database for Analytics”. In: *VLDB*. 2022.

[15] Konstantinos Sioulas et al. “Hardware-Conscious Hash-Join on GPUs”. In: *IEEE ICDE*. 2019.

[16] Farzad Khorasani et al. “Efficient Hash Join on GPUs”. In: *IEEE ICDE*. 2018.

[17] Runyu Lu, Xixian Qi, et al. “Rummy: GPU-Based Approximate Nearest Neighbor Search”. Manuscript under review. Dec. 2025.

[18] Ruiqi Guo et al. “Quantization for Fast Similarity Search”. In: *NeurIPS*. 2020.

[19] John L. Hennessy and David A. Patterson. “Computer Architecture: A Quantitative Approach”. In: (2011).

[20] Zhe Wang et al. “Taurus: An Efficient and Scalable FPGA-Accelerated Database System”. In: *ACM SIGMOD*. 2022.

[21] Raj Prabhakar et al. “Plasticine: A Reconfigurable Architecture For Parallel Patterns”. In: *ISCA*. 2017.

[22] Lanyue Lu et al. “WiscKey: Separating Keys from Values in SSD-Conscious Storage”. In: *USENIX FAST*. 2016.

[23] Graham Cormode and S. Muthukrishnan. “An Improved Data Stream Summary: The Count-Min Sketch and Its Applications”. In: *LATIN*. 2004.

[24] Brian F. Cooper et al. “Benchmarking Cloud Serving Systems with YCSB”. In: *ACM SoCC*. 2010.

[25] Xixian Qi et al. “HiStore: Rethinking Hybrid Storage for Key–Value Stores”. Manuscript in preparation. June 2026.

[26] Xixian Qi et al. “GPU-Based Hash Join with Partition Pruning”. Manuscript in preparation. June 2026.

[27] Jiong He, Mian Lu, and Bingsheng He. “Revisiting Co-Processing for Hash Joins on the Coupled CPU-GPU Architecture”. In: *VLDB*. 2013.

[28] Xixian Qi, Runyu Lu, et al. “Hitcher: Efficient GPU-Based Vector Search via Cluster-Centric Kernel and Hitch-Ride Ordering”. In: *Proceedings of the 2026 ACM SIGMOD International Conference on Management of Data*. Accepted; to appear. 2026.